

Manual de Implementación

Voluntariando

Plataforma de gestión de voluntariado comunitario

Guía paso a paso para personas sin conocimientos de programación

Creado por Claude.ai con ayuda (mínima)
de Víctor González (vicjgf@gmail.com)

Monterrey, Nuevo León, México · 2026

Contenido

Contenido	2
1. ¿Qué vas a construir?	3
1.1 Lo más importante que debes saber	3
1.2 Qué necesitas antes de empezar	3
2. La arquitectura, explicada sin tecnicismos	3
3. Paso 1 — Obtener el código	4
4. Paso 2 — Firebase (datos y usuarios)	4
4.1 La llave de servicio (el paso delicado)	5
4.2 Las reglas de seguridad	5
5. Paso 3 — GitHub (publicar el código)	5
6. Paso 4 — Cloudflare (el servidor)	5
6.1 Personalizar la configuración	6
6.2 La llave secreta (el paso donde todos se atoran)	6
7. Paso 5 — Conexiones finales	6
7.1 Autorizar tu dominio en Firebase	6
7.2 Habilitar los proveedores de acceso	7
8. El modo demostración (¡importante!)	7
8.1 Cómo pasar al modo seguro	7
9. Paso 6 — Personalizar tu plataforma	7
9.1 Poner tu logo	7
9.2 Cambiar los colores y textos	8
10. Paso 7 — Pruebas antes de lanzar	8
11. Problemas comunes y sus soluciones	8
12. Costos y límites: ¿hasta cuándo es gratis?	8
13. Cómo trabajar con un asistente de IA (el secreto de todo)	9
14. Glosario	9

1. ¿Qué vas a construir?

Esta guía te enseña a crear una plataforma web para organizar voluntariado en tu comunidad. Una organización publica eventos (jornadas de limpieza, brigadas, campañas) y cualquier persona se inscribe a las actividades desde su celular, sin tener que crear una cuenta.

Cada evento puede tener varias actividades con cupos limitados. Los coordinadores gestionan todo desde un panel protegido: crean eventos, eligen qué datos pedir a los voluntarios, ven la lista de inscritos y la exportan. El público solo ve nombres de pila y cuántos lugares quedan; los datos personales (correos, teléfonos) quedan protegidos.

1.1 Lo más importante que debes saber

- **No necesitas saber programar.** Este manual nació de la experiencia de un trabajador social sin conocimientos técnicos que construyó la plataforma conversando con un asistente de inteligencia artificial (Claude, de Anthropic). Tú harás lo mismo: crear cuentas, copiar, pegar y pedir ayuda a la IA cuando algo se complique.
- **Cuesta \$0.** Toda la plataforma corre sobre servicios gratuitos: Cloudflare (el servidor), Firebase (la base de datos y los usuarios) y GitHub (donde vive el código). Los límites gratuitos alcanzan para comunidades de cientos o miles de personas. Claude tiene límites de uso diarios y es posible que te impida continuar en cierto momento (pidiéndote que te suscribas). Basta con esperar unas horas para poder continuar trabajando gratis.
- **Es más segura que muchas apps comerciales.** Está diseñada para que los datos personales de los voluntarios nunca queden expuestos. Más adelante te explicamos cómo lo logra.
- **Tomará entre 1 y 2 días de trabajo,** distribuidos en sesiones cortas. La mayor parte del tiempo se va en crear cuentas y conectar servicios, no en cosas técnicas complejas.
- **El código ya existe.** No vas a escribir código: vas a copiar el proyecto Voluntariando (que es público en <https://github.com/vicjgf/voluntariando>) y personalizarlo con cambios mínimos.

1.2 Qué necesitas antes de empezar

- Una computadora con navegador Google Chrome (las configuraciones se hacen mejor en computadora; el sitio final funcionará perfecto en celulares).
- Un teléfono celular para recibir códigos al crear cuentas.
- De preferencia, acceso a un asistente de IA (Claude en claude.ai, o similar) para pedir ayuda cuando algo no salga como dice el manual. Los servicios web cambian sus pantallas con frecuencia.
- Tu correo de Gmail (servirá para crear todas las cuentas y para ser el administrador).

 *Regla de oro: crea UN correo de Gmail exclusivo para el proyecto (por ejemplo: voluntariado.tucomunidad@gmail.com) y úsalo para TODAS las cuentas (Google, GitHub, Firebase, Cloudflare). Así el proyecto no depende del correo personal de nadie y se puede transferir la administración en el futuro.*

2. La arquitectura, explicada sin tecnicismos

Tu plataforma se compone de tres piezas gratuitas que se conectan entre sí. Entender qué hace cada una te dará seguridad cuando configures o cuando algo falle:

Pieza	Qué es	Analogía
GitHub	Donde vive el código (los archivos del proyecto). Es el "archivo maestro".	El plano de la casa
Cloudflare	El servidor que corre el sitio y lo muestra al público. También es el "guardián" que protege los datos.	La casa y su portero
Firebase	Guarda los datos (eventos, inscripciones) y maneja el inicio de sesión de los coordinadores.	El archivero y la cerradura

La gran diferencia de esta plataforma frente a otras más sencillas es que el navegador del visitante NUNCA toca la base de datos directamente. Todo pasa por Cloudflare (el portero), que decide qué información entregar según quién pregunta:

- **Un visitante** solo recibe nombres de pila y el conteo de lugares.
- **Un coordinador** que inició sesión recibe los datos completos (correos, teléfonos).

Así, aunque alguien con conocimientos técnicos intente "espiar" la base de datos, no encontrará nada: está cerrada con llave y solo el portero tiene acceso.

3. Paso 1 — Obtener el código

El proyecto Voluntariado es público y funciona como plantilla. Así lo descargas:

1. Entra al repositorio público de Voluntariado en GitHub (<https://github.com/vicjgf/voluntariado>).
2. Haz clic en el botón verde Code y luego en Download ZIP.
3. Descomprime el ZIP en una carpeta de tu computadora. Verás estos archivos:

```
src/worker.js      ← el "portero" (servidor)
public/index.html  ← la página que ve la gente
wrangler.toml      ← LA CONFIGURACIÓN (aquí harás tus cambios)
package.json       ← datos del proyecto
README.md          ← guía técnica resumida
```

⚠ El proyecto trae la configuración del proyecto original. NO la uses: en los siguientes pasos crearás la tuya propia y la reemplazarás. Usar la de otro proyecto haría que tus datos se mezclen con los suyos (y la seguridad lo impediría de todos modos).

4. Paso 2 — Firebase (datos y usuarios)

Firebase es de Google. Con el correo del proyecto:

1. Entra a console.firebase.google.com y crea un proyecto (ejemplo: voluntariado-tucomunidad). Puedes desactivar Google Analytics.
2. Activa Firestore Database → Crear base de datos → modo Producción → región us-central (o la más cercana).
3. Activa Authentication → pestaña "Sign-in method" → habilita Google. Habilita también Correo electrónico/Contraseña y, dentro de esa opción, activa el segundo interruptor Vínculo de correo (sin contraseña).

4. Ve a  Configuración del proyecto → General → baja a "Tus apps" → crea una app Web (icono </>). Te mostrará un bloque llamado firebaseConfig con 6 datos. Cópialos y guárdalos — los pegarás en el código.

4.1 La llave de servicio (el paso delicado)

El "portero" (Cloudflare) necesita una llave especial para poder hablar con la base de datos de forma segura. Esta llave es secreta, como la contraseña maestra de tu archivero.

1. En Firebase:  Configuración del proyecto → pestaña Cuentas de servicio.
2. Haz clic en Generar nueva clave privada. Se descargará un archivo que termina en .json.
3. Guárdalo en un lugar seguro de tu computadora.

⚠ Ese archivo .json es la llave maestra de tu base de datos. NUNCA lo subas a GitHub ni se lo compartas a nadie (tampoco a la IA). Si alguna vez se expone, genera una nueva llave de inmediato.

4.2 Las reglas de seguridad

Las reglas determinan quién puede leer y escribir en tu base de datos. Como aquí el único que entra es el "portero", cerramos TODO el acceso directo. Ve a Firestore → pestaña Reglas, borra lo que haya, pega esto y publica:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

💡 ¿Te asusta poner "todo cerrado"? Es justo lo correcto aquí. El portero (Cloudflare) usa la llave de servicio, que pasa por encima de estas reglas. La gente entra por el portero, nunca directo a la base de datos.

5. Paso 3 — GitHub (publicar el código)

1. Crea una cuenta en github.com con el correo del proyecto.
2. Crea un repositorio nuevo: botón New, nombre corto y sin espacios (ejemplo: voluntariado-tucomunidad), Público, sin README.
3. En el repo: Add file → Upload files. Arrastra TODO el contenido de la carpeta: las carpetas src y public completas, y los archivos sueltos (wrangler.toml, package.json, README.md). Commit.

⚠ Asegúrate de que las carpetas src y public se suban CON sus archivos adentro. Los archivos que empiezan con punto (como .gitignore) están ocultos en Windows/Mac; no pasa nada si no se suben, no son indispensables.

6. Paso 4 — Cloudflare (el servidor)

1. Crea una cuenta gratuita en dash.cloudflare.com con el correo del proyecto.

2. Ve a Workers & Pages → Create.
3. Conecta tu repositorio de GitHub (autoriza el acceso cuando te lo pida).
4. En la configuración del build, deja el Build command vacío y el Deploy command como:
npx wrangler deploy
5. Despliega. Cloudflare te dará una dirección web tipo tu-proyecto.workers.dev.

6.1 Personalizar la configuración

Antes o después del primer despliegue, abre el archivo wrangler.toml (con el Bloc de notas) y cambia estos tres valores por los de TU proyecto Firebase. Vienen en tu archivo .json y en la firebaseConfig que copiaste:

```
[vars]
FIREBASE_PROJECT_ID = "tu-proyecto-id"
FIREBASE_CLIENT_EMAIL = "firebase-adminsdk-...@tu-proyecto.iam.gserviceaccount.com"
ADMIN_EMAIL = "tucorreo@gmail.com"
```

También pega tu firebaseConfig en el archivo public/index.html, en el bloque marcado con "SI REPLICAS ESTE PROYECTO".

💡 Si editar archivos te intimida, pídeselo a la IA: "Aquí está mi wrangler.toml y mi index.html, y estos son mis datos de Firebase [pégalos, MENOS la llave privada]. Devuélveme los archivos ya configurados".

6.2 La llave secreta (el paso donde todos se atoran)

Las tres variables de arriba van en el archivo wrangler.toml. Pero la llave privada (que es secreta) NO debe ir en un archivo público. Se pone aparte, directamente en Cloudflare:

1. En Cloudflare, entra a tu Worker → Settings → Variables and Secrets (la sección que dice "used at runtime").
2. Haz clic en Add. Nombre: FIREBASE_PRIVATE_KEY. Valor: el campo private_key de tu archivo .json, completo, SIN las comillas de los extremos. Marca la opción Encrypt (Secret).
3. Guarda.

⚠ MUY IMPORTANTE: las tres variables del wrangler.toml se reponen solas en cada despliegue (porque están en el archivo). La llave secreta también debería sobrevivir, pero si algún día el sitio deja de funcionar de repente, lo PRIMERO que debes revisar es que las 4 variables sigan ahí. En las pruebas, las variables se borraban en cada despliegue hasta que las pusimos en el wrangler.toml.

💡 Truco de diagnóstico: si tienes dudas de si las variables llegaron, pídele a la IA que agregue temporalmente un "endpoint de diagnóstico" que te diga cuáles están presentes sin revelar sus valores. Recuerda pedirle que lo borre después.

7. Paso 5 — Conexiones finales

Dos amarres indispensables para que el inicio de sesión funcione:

7.1 Autorizar tu dominio en Firebase

Ve a Firebase → Authentication → Settings → Dominios autorizados → Agregar dominio. Escribe la dirección de tu Worker SIN el https:// ni la barra final. Ejemplo:

7.2 Habilitar los proveedores de acceso

Vuelve a confirmar en Firebase → Authentication → Sign-in method que estén habilitados Google y Correo electrónico/Contraseña con el vínculo de correo activado. Es el olvido más común al crear un proyecto nuevo.

8. El modo demostración (¡importante!)

La plantilla viene configurada en MODO DEMOSTRACIÓN para que cualquiera pueda probarla sin pedir permiso. En este modo:

- **Sin iniciar sesión** → eres visitante (ves eventos y te inscribes).
- **Al iniciar sesión** (con cualquier Google o correo) → te conviertes en coordinador y puedes crear, editar y borrar eventos libremente.

Esto es ideal para que la gente experimente. Pero NO es seguro para una instalación real, porque cualquiera podría modificar tus eventos.

8.1 Cómo pasar al modo seguro

Cuando hagas tu instalación de verdad, haz estos dos cambios:

1. En el archivo `src/worker.js`, busca el bloque marcado "MODO DEMO ACTIVADO" y borra la línea que dice:

```
return 'coordinador'; // ← BORRA ESTA LÍNEA PARA EL MODO SEGURO
```

1. En el archivo `public/index.html`, borra el bloque del banner "Modo demostración" (está marcado con comentarios) y, si quieres, las palabras "(DEMO Mode)" del título.

Con eso, solo los correos que tú (el administrador) agregues a la lista de coordinadores — desde el botón  Coordinadores dentro de la app— tendrán permisos de edición. El resto serán visitantes.

 Recuerda volver a subir a GitHub los archivos que cambies. Cloudflare los redespliega solo en 1-2 minutos.

9. Paso 6 — Personalizar tu plataforma

9.1 Poner tu logo

La plantilla trae un placeholder que dice "Tu logo aquí" en dos lugares: el encabezado y la pantalla de inicio de sesión. Para poner el tuyo:

1. Sube tu logo a un servicio gratuito como imgur.com o postimages.org y copia el enlace directo de la imagen (termina en `.png` o `.jpg`).
2. En `public/index.html`, busca el comentario "LOGO del encabezado" y reemplaza la línea del emoji por:

```

```

1. Busca el bloque "Tu logo aquí" de la pantalla de login y reemplázalo por:

```

```

💡 Un logo PNG con fondo transparente se ve mejor sobre el color de fondo. Para el login, un logo cuadrado o circular luce mejor; para el encabezado, uno horizontal.

9.2 Cambiar los colores y textos

En `public/index.html`, al inicio de la sección de estilos, está la paleta de colores. Cambia los códigos (formato hexadecimal, como `#2ec4b6`) por los de tu organización. También puedes buscar y reemplazar la palabra "Voluntariando" por el nombre de tu iniciativa.

10. Paso 7 — Pruebas antes de lanzar

Verifica esta lista, de preferencia desde un celular y también desde computadora:

- Ver los eventos sin iniciar sesión (como visitante).
- Iniciar sesión con Google y con correo (link mágico — puede caer en SPAM, es normal).
- Crear un evento con varias actividades y campos.
- Inscribirte como visitante y confirmar que aparece tu nombre de pila pero no tu correo.
- Como coordinador, ver la lista completa de inscritos y exportarla.
- Probar los estados: desactivar un evento, marcarlo lleno, ocultar una actividad.
- Borrar un inscrito y un evento.

11. Problemas comunes y sus soluciones

Todos estos nos pasaron al construir la plataforma. Te ahorramos el susto:

Problema	Solución
"Error al iniciar sesión" con Google	Falta autorizar el dominio en Firebase → Authentication → Settings → Dominios autorizados. O no habilitaste Google como proveedor (sección 7.2).
El sitio responde con error 500 o no carga eventos	Casi siempre es la llave privada (<code>FIREBASE_PRIVATE_KEY</code>) que no llegó o está mal pegada. Revisa que las 4 variables estén en Cloudflare (sección 6.2).
Error 401 al entrar (no autorizado)	Falta la variable <code>FIREBASE_PROJECT_ID</code> , o no coincide exactamente con tu proyecto. Verifícala en el <code>wrangler.toml</code> .
Las variables se borraron solas tras un despliegue	Las 3 públicas deben estar en el <code>wrangler.toml</code> para que se repongan solas. Si se borró la llave secreta, agrégala de nuevo en Variables and Secrets.
El link de correo no llega	Revisa la carpeta de SPAM. El correo viene de un dominio de Firebase y a veces cae ahí.
Subí cambios y el sitio no se actualiza	Cloudflare tarda 1-2 minutos en redespargar. Recarga con <code>Ctrl+Shift+R</code> o en una ventana de incógnito.
Cualquiera puede crear y borrar eventos	Es el modo demo. Para cerrarlo, ve a la sección 8.1 de este manual.

12. Costos y límites: ¿hasta cuándo es gratis?

Con los planes gratuitos, los límites son muy holgados para una comunidad:

- **Cloudflare:** 100,000 peticiones por día. Imposible de agotar para uso comunitario.
- **Firebase (plan Spark):** 50,000 lecturas de base de datos por día. Una iniciativa hiperlocal tarda meses en acercarse.

Si algún día creces de verdad, el plan de pago de Firebase cuesta centavos, y cualquier asistente de IA puede ayudarte a optimizar el código llegado el momento.

13. Cómo trabajar con un asistente de IA (el secreto de todo)

Esta plataforma fue construida íntegramente por una persona sin conocimientos de programación conversando con Claude. Estos son los aprendizajes de ese proceso:

- **Trabaja paso a paso.** Pide una cosa a la vez y confirma que funciona antes de la siguiente. "Hazme toda la plataforma" produce errores; "ahora agreguemos el botón de inscripción" produce resultados.
- **Comparte capturas de pantalla.** Cuando algo falle, sube una captura de lo que ves (incluyendo la consola del navegador: tecla F12 → pestaña Console). La IA diagnostica mucho mejor viendo el error exacto.
- **Pide que te pregunte.** Antes de cambios grandes, di: "hazme preguntas para asegurarte de que entendiste mis ideas". Evita malentendidos costosos.
- **Pide auditorías.** Cada cierto tiempo: "evalúa mi sitio con honestidad, sin complacencia: ¿qué le falta?, ¿qué riesgos tiene?". Así se detectaron problemas de seguridad importantes en este proyecto.
- **Nunca compartas la llave privada.** Puedes compartir tu configuración pública, pero el archivo .json de la cuenta de servicio jamás. Ni a la IA.
- **Desconfía amablemente.** Si una instrucción de la IA no coincide con lo que ves en pantalla, díselo. Las interfaces de Google, GitHub y Cloudflare cambian seguido y la IA puede tener información desactualizada.

14. Glosario

Término	Qué significa
Backend	La parte del sistema que corre en el servidor y que el usuario no ve. Aquí, el "portero" (Worker) es el backend.
Cloudflare Worker	Un pequeño programa que corre en los servidores de Cloudflare. Es el "portero" que protege los datos.
Commit	Guardar un cambio en GitHub. Cada commit queda registrado en el historial.
Despliegue (deploy)	El acto de publicar el código para que el sitio funcione con la última versión.
Endpoint	Una "puerta" del servidor a la que el sitio le hace preguntas (por ejemplo, "dame los eventos").
Firestore	La base de datos de Firebase donde se guardan eventos e inscripciones.
Frontend	La parte que el usuario ve: la página web con sus botones y colores.

Término	Qué significa
Repositorio (repo)	La carpeta del proyecto en GitHub, con todo su código e historial.
Service account	La "llave de servicio": credenciales que permiten al portero hablar con la base de datos.
Variable de entorno	Un dato de configuración que se guarda fuera del código (como las llaves), por seguridad.